

A group of people are working in a modern office environment. In the foreground, a man with a beard and glasses is wearing large headphones and looking at a laptop. To his right, another man is also working on a laptop. In the background, a woman is visible, and another man is partially seen on the far right. The office has large windows and a modern chandelier with multiple glass shades hanging from the ceiling.

Recourse

An automated HR advisor

CV screening that **explains itself** — to both sides

Gleb Tretiakov · Vitalii Virronen · Ilia Zaleskii

AaltoAI Hackathon 2026 — AI sovereignty, security & EU data law

recourse.ilia.fi · open source

250 applications. One afternoon.

Generative AI made applying free.
It did not make **reading** free.

Two people are stuck

The HR manager

- Hundreds of CVs per opening
- No time to read them all properly
- Buys a black-box filter → now **legally exposed**
- Cannot answer "why was I rejected?"

The applicant

- Sends 100 applications
- Gets silence, or "we moved forward with other candidates"
- Learns **nothing** — cannot improve, cannot contest
- Reapplies next year with the same CV

The problem, in EU AI Act terms. AI that screens or ranks job applicants is a **high-risk system** (Reg. 2024/1689, Annex III). Whoever deploys one must keep a human in charge of the decision, log every decision so it can be audited, and explain each one to the person affected.

A black-box filter delivers none of the three — and every rejection it issues is a decision it cannot account for.

Our bet

Tell the rejected candidate
exactly what would have changed the answer.

"One more shipped microservice project would have been enough."

The technique is called **algorithmic recourse** — well studied in papers,
almost absent from shipped hiring products.

What we built

The screenshot displays the Recourse interface for a job titled "Hiring for Backend Software Engineer". At the top, there are filters for "A - fixed threshold 80" and "B - top N of pool" (N=3). The left sidebar shows "Uploaded CVs" with a list of candidates, including Priya Sharma and Daniel Khan. The main content area is divided into two sections: "Why this match?" and "Candidate Email".

Why this match? section for Priya Sharma:

- Reconstructed from the filename. The screening model never sees it — protected attributes are not representable in the profile at all.
- Score: 61/100 (NOT ADVANCED) 4.0 yrs experience
- Rank: 6 of 12 - top 3 advance - bar to beat 91
- Not advanced at 61/100 against a bar of 91. Strongest: professional backend engineering experience (32 pts). Largest shortfall: hands-on Kubernetes experience (deploying and operating a service) (~11 pts). Ranked 6 of 12; top 3 advance.
- HARD REQUIREMENTS:** Evaluated before any weighted score, and never traded off against it.
 - skills.python_held == true
 - experience.software_months >= 12
 - education.highest_level >= bsc
- MATCHED REQUIREMENTS:**
 - professional backend engineering experience
 - professional Python experience
 - completed degree level
 - working knowledge of Docker
 - working knowledge of SQL databases
- GAPS IDENTIFIED:**
 - hands-on Kubernetes experience (deploying and operating a service) ~11
 - shipped microservices projects ~10

Candidate Email section:

- To: priya.sharma@example.com (synthetic — no address found in the CV)
- Update on your application — Backend Software Engineer
- LLM sentences checks passed explain-vi/claude-opus-5
- Hi Priya,
- Your application did not move forward at this stage.
- Here is some feedback to help you succeed (this would have allowed you to advance to the interview stage in this hiring round):
- Route 1 (build):
 - Take on work using AWS, Azure or GCP directly, which typically takes about three months to build up.
 - Ship one microservices project and list it on your CV; this usually takes around two months.
 - Get hands-on with Kubernetes by deploying and operating a service, which typically takes about four months.
- This route would have placed you around rank 3 in this pool.
- Buttons: Send, Keep further

Top bar — the job being hired for, and the bar to clear: the **top N of the pool** advance (N = 3 here). Settings open the job editor and the audit log.

Uploaded CVs — drop in CVs, get a ranked shortlist. Each candidate carries a score out of **100**; the job is authored as a 100-point budget, so the score reads as a percentage.

Why this match? — hard requirements ticked off, matched requirements, and the gaps with their point values. Shows which parts of the CV drove the score. Approve or override every decision.

Candidate email — up to **3 independent routes** back to a pass, each with a cost and a realistic timeline, pre-drafted from the solver's output. The manager edits and sends. Never sent automatically.

1 • Parse

LLM reads the CV, emits a structured profile. Cached by content hash.

2 • Score

Deterministic. Knockouts first, then the 100-point budget.

3 • Recourse

CP-SAT solver finds the cheapest changes that flip the outcome.

4 • Explain

Code-owned templates; the LLM may rephrase, never invent.

5 • Log

Hash-chained, append-only audit record per decision.

The LLM appears **twice** (steps 1 and 4); everything between is deterministic and replayable. Open source · self-hostable · runs the whole demo with **no LLM key**.

The job is a point budget

HR authors it. The system enforces it. The column sums to **100**, so a score reads as a percentage and every candidate in the pool is measured on the same scale.

Hard requirements — knockouts

Fail one → rejected. No score traded off against it.

```
skills.python.held == true
experience.software_months >= 12
education.highest_level >= bsc
```

Soft requirements — 100 points

Criterion	Pts
Backend experience	32
Python	16
Kubernetes	11
Microservices projects	10
Cloud platform	9
Education level	6
CI/CD · Docker · IaC · SQL	5+4+4+3

Recourse is an optimisation problem

Not a prompt.

Given the candidate's profile, find the **cheapest set of changes** that crosses the bar:

Actionability Every feature is tagged

`actionable` , `conditionally actionable` or `immutable` .

Immutable features are never in a route.

Cost & time Each step has a cost and a typical duration, from the job-family manifest — not invented per candidate.

Causal constraints

$k8s \leq \text{docker}$

$\Delta_{\text{micro}} \leq 2 + \Delta_{\text{backend_mo}}$

You cannot be advised to fake a dependency.

CP-SAT returns up to **3 diverse routes**, each sufficient on its own, sparsity-penalised so short routes win.

Law compliance

Hiring AI is **high-risk under Annex III** of the EU AI Act. We designed to the strict reading.

EU AI Act (2024/1689) Art. 86 — right to an explanation of an individual decision. → every decision carries its contributions, its routes and its config version.

Art. 12 — record-keeping: the system must **log** its decisions. → an append-only, **hash-chained** audit log. Each record stores the profile hash, job version, scorer and solver versions, model id and the routes shown; `make audit` verifies the chain. High-risk duties deferred to 2 Dec 2027 (Digital Omnibus 2026/1744).

GDPR Art. 22 No solely-automated decision with significant effect.
→ **the manager decides**. The system shortlists and drafts; a human sends.

GDPR Art. 13–15 — "meaningful information about the logic involved." → the logic *is* the published point budget.

Data minimisation → protected attributes are never extracted into the profile; the model scores a feature vector, not a person.

NIS2 / sovereignty → self-hostable, two containers, no third-party ATS.

Demo

1. Open the pool — 10 candidates, ranked
2. Open **Aisha** — 75/100, rejected
3. Read the three routes
4. Open the email draft, polish it, *don't* send
5. Open the job editor — move Kubernetes from 11 pts to 0, re-screen

recourse.ilia.fi

Recourse

An automated HR advisor

Screening that can explain itself — to the manager **and** to the candidate

Gleb Tretiakov · Vitalii Virronen · Ilia Zaleskii

recourse.ilia.fi · open source · AaltoAI Hackathon 2026

Appendix

Score math · Honesty rules · Keeping the model on a leash

Score math

Each criterion has a **cap** — how many solver steps count as full marks — so one step is worth `points // cap`, always an integer.

```
score = Σ min(steps(feature), cap) × (points // cap)
```

The bar — top N of the pool A candidate advances if they are in the **top 3**. The bar to beat is the 3rd-best score in the pool, so recourse is computed against the actual competition, not a fixed number.

Every route the solver returns is enough to clear that bar on its own, given the rest of the pool as it stands today.

Honesty rules, in code

The system must never claim the candidate lacks something it merely **didn't see**.

stated / computed

→ **instruction**

"Add about 6 more months of professional software engineering experience."

denied

→ **instruction that concedes**

"Your CV said you do not have hands-on Kubernetes experience, so gaining it is one way to close this gap."

absent

→ **question**

"Your CV did not mention infrastructure-as-code — if you have it, add it and ask us to re-run the screen."

Verbatim output from the running system.

Keeping the model on a leash

What the LLM writes

- One sentence per change
- That is all

What code writes

- The framing and intro
- Ordering of routes
- Immutable-blocker disclosures
- The closing text

What checks it

A **code-only checker** runs on every generated sentence:

- whitelist of facts it may mention
- no new numbers, no new features
- fails → fall back to the template

No LLM judges another LLM. No "as an AI language model" apology path.

Every screening in this demo ran with `explain=false` — the deterministic path.